EECS484 Mid

- External Schema
- Conceptual/Logical ~
- Physical Schema

- DDL: Data Def. Lan
- DML: Data Man. Lan

ER Diagram

- Entity ▭
- Attribute ◯
- Relation ◇
- ISA hiracy (isA)
- Aggregation

Constraints

key constraint: → (at most one)

participation ~: — (at least one)

→ (exactly one)

e.g. [child] → (has) — [mother]

keys

- primary key
- candidate key
- foriegn key
- minimal key
- super key

Relational Algebra

- Selection σ
- projection π
- cross-product $\times$
- set-difference $-$
- union $\cup$
- intersection $\cap$
- join $\bowtie$ $\bowtie_c$
  - $R \bowtie_c S = \sigma_c(R \times S)$
- division $/$
  - $A/B = \pi_x(A) - \pi_x((\pi_x(A) \times B - A)$
  - some attribute in A, but is not in B.
- renaming ρ

e.g. Reserves (sid, bid, day) — Sailors (sname, sid, rating, age) — Boats (bid, color)

* Find Sailors who reserved Boat #103

$\pi_{sname}((\sigma_{bid=103}$ Reserves $\bowtie$ Sailors$)$

* Find sailors who reserved all boat

$\rho(Temp, (\pi_{sid,bid}$ Reserves$) / (\pi_{bid}$ Boats$))$

$\pi_{sname}(Temp \bowtie$ Sailors$)$

SQL Basics

```
CREATE TABLE Users(
    uid NUMBER PRIMARY KEY,
    name VARCHAR2(100) NOT NULL,
    bdate INTEGER
    UNIQUE(name)
);

CREATE TABLE Friends(
    uid1 NUMBER,
    uid2 NUMBER,
    PRIMARY KEY (uid1, uid2),
    FORIEGN KEY (uid1)
      REFERENCES Users(uid)
      ON DELETE CASCADE,
    ……
    CHECK(uid1 <> uid2)
);

INSERT INTO Users(
    uid, name, bdate)
VALUES (123, 'Sindy', 117);
DELETE FROM Users U
WHERE U.uid = 123;
ALTER TABLE Users
ADD COLUMN age: INTEGER;
DROP TABLE
Users;

UPDATE Users SET
    name='Qinye'
    WHERE uid=123;
CREATE VIEW myView(
    uid, name) AS
SELECT
    CAST(U.uid AS
      VARCHAR2(100)),
    U.name
FROM Users U;
```

Query

```
e.g. SELECT S.name
     FROM Sailors S, Reserves R,
     WHERE S.sid=R.sid
       AND R.bid=103;
     SELECT S.name
       FROM Sailors S
     Reserves R ON S.sid=R.sid
     WHERE R.bid=103;
```

NATURAL JOIN

- INNER JOIN (default)
- OUTER JOIN
  - LEFT JOIN
  - RIGHT JOIN
  - FULL JOIN

Set operators

- UNION (eliminate dups)
- UNION ALL
- INTERSECT
- MINUS/EXCEPT

```
e.g. SELECT …
     UNION SELECT …
```

Aggregate Operators

- COUNT
- SUM
- AVG
- MAX
- MIN

```
e.g. SELECT S.name,
       S.age FROM
     Sailors S
     WHERE S.age=
       (SELECT MAX(S2.age)
       FROM Sailors S2);
```

Nested Queries

- attr IN R
- EXISTS R
- UNIQUE R
- NOT …

Group By

```
e.g. SELECT S.rating
       MIN(S.age)
     FROM Sailors S
     WHERE S.age>=18
     GROUP BY S.rating
     HAVING COUNT(*)>=2
     ORDER BY S.rating
       [ASC]/[DESC]
```

NULL (Unknown)

U AND T = U

U AND F = F

U OR T = T

U OR F = U

Normalization

Functional Dependency

$D: X \to Y$. Then $\forall t_1, t_2 \in r$

$\pi_x(t_1) = \pi_x(t_2)$

$\Rightarrow \pi_y(t_1) = \pi_y(t_2)$

Armstrong's Inference Axioms

#1 Reflexive property → trivial dependency

#2 Transitivity

$X \to Y, Y \to Z \Rightarrow X \to Z$

#3 Augmentation

$X \to Y \Rightarrow XZ \to YZ$

```
SELECT S.name, S.age
FROM Sailors S
WHERE S.age >=
  ALL (SELECT S2.age
    FROM Sailors S2)
```

ANY

Union $X \to Y$. $X \to Z$
$\Rightarrow X \to YZ$

Decomposition
$X \to YZ \Rightarrow X \to Y$. $X \to Z$

closure of F: F+

Decomposition

Lossless Decomposition
* Intersection must be a super key to one of the relations

Dependency Preserving ~

BCNF FDs must be
* trivial
or * left side must be a super key

3NF FDs must be
* trivial
or * left side is a super key
or * left side is a part of minimal key

Note: Decomposition into 3NF is always gauranteed.

e.g. $BC \to D$ (i)
$CD \to B$ (ii)
$D \to E$ (iii)
$ACD \to F$ (iv)

keys: ABC. ACD.

FDs violating BCNF: (i) (ii) (iii)

FDs violating 3NF: (iii)

e.g. Booktown

* Find authors who have written books in any given subjects.

```
SELECT A.aid
FROM Books B, Authors A
WHERE A.aid=B.aid
  AND B.aid NOT IN(
    SELECT B2.aid
    FROM Books B2,
      Subjects S
    WHERE(B.aid,S.sid)
      NOT IN(
        SELECT B3.aid,
          B3.sid
        FROM Books B3
      )
);
```

* Find authors who wrote no books

```
SELECT A.aid
FROM Authors A
WHERE A.aid NOT IN(
    SELECT DISTINCT B.aid
    FROM Books B
);
```

* Find authors who wrote exactly 1 book

```
SELECT A.aid, A.name
FROM Authors A, Books B
WHERE A.aid = B.aid
GROUP BY A.aid, A.name
HAVING COUNT(*)=1
ORDER BY A.aid ASC;
```

e.g. Classes & Students

* Find students taking 484.482 or 584.582

```
SELECT DISTINCT S.sid
FROM Students S
  Courses C1, Courses C2
  Enrolled E1, Enrolled E2
WHERE S.sid=C1.sid
  AND S.sid=C2.sid
  AND C1.sid=E1.sid
  AND C2.cid=E2.cid
  AND((C1.title='EECS482'
  AND C2.title='EECS484')
  OR(C1.title='EECS582'
  AND C2.title='EECS584'));
```

* Find CS-major students in non-CS heavy courses

```
SELECT DISTINCT S1.sid
FROM Student S1
  Enrolled E1
WHERE S1.sid=E1.sid
  AND S1.major='CS'
  AND E1.cid IN(
    SELECT E2.cid
    FROM Enrolled E2,
      Students S2
    WHERE S2.sid=E2.sid
    AND S2.major<>'CS'
    GROUPY E2.cid
    HAVING COUNT(*)>50)
ORDER BY S1.sid DESC;
```

* Find Student Pairs who take the same course but are not project partners

```
CREATE VIEW
  StudentPairs AS
SELECT DISTINCT
  E1.sid, E2.sid
FROM Enrolled E1,
  Enrolled E2
WHERE E1.cid=E2.cid
  AND E1.cid<E2.sid
MINUS
  SELECT DISTINCT
    M1.sid, M2.sid
  FROM Member M1,
    Member M2
  WHERE M1.pid=M2.pid
  AND M1.sid<M2.sid
```